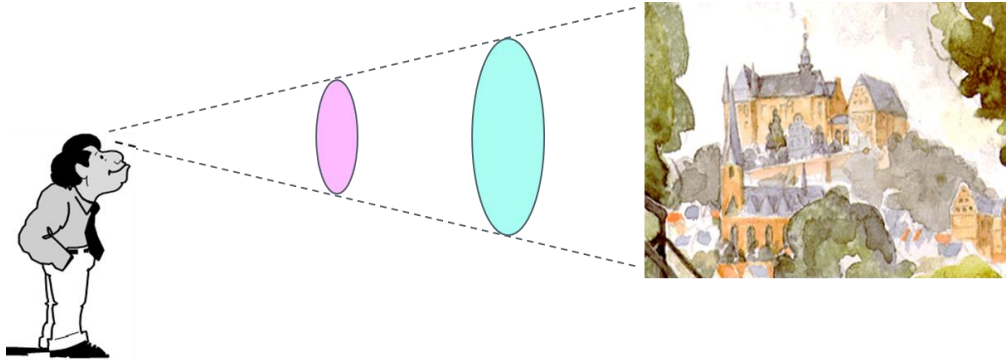


Modellgetriebene Softwareentwicklung

Benjamin Hoffmann, M. Sc.
Marco Richter, M. Sc.

Kapitel 01

EINFÜHRUNG MDSD – MODELLE, KONZEPTE, BEISPIELE



- Menschen besitzen die Fähigkeit zur Abstraktion, d.h. Erstellung eines mentalen Modells der Realität (durch Erkennen von Gemeinsamkeiten einer Reihe von Sinneseindrücken)
 - Generalisierung bestimmter Eigenschaften von realen Objekten
 - Klassifizierung von Objekten zu Gruppen
 - Aggregation von simplen Objekten zu komplexeren Objekten.

vgl. Brambilla, Marco; Cabot, Jordi; Wimmer, Manuel (2012): Model-driven software engineering in practice. p. 1

Definition: Modell

"(Wissenschaft) Objekt, Gebilde, das die inneren Beziehungen und Funktionen von etwas abbildet bzw. [schematisch] veranschaulicht [und vereinfacht, idealisiert]"

Duden, <https://www.duden.de/rechtschreibung/Modell>

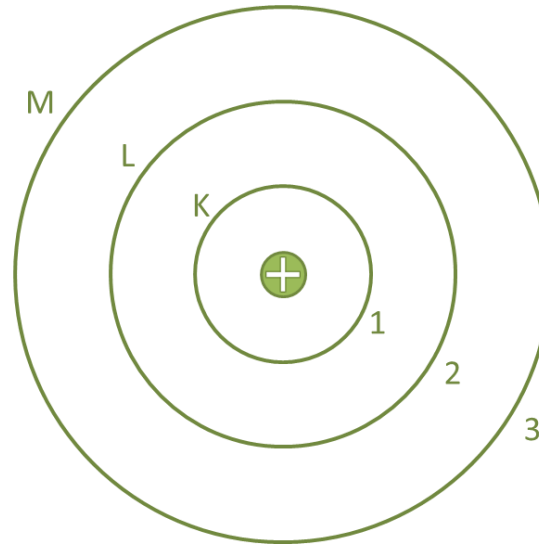
"We can informally define a model as a simplified or partial representation of reality, defined in order to accomplish a task or to reach an agreement on a topic. Therefore, by definition, a model will never describe reality in its entirety."

Brambilla et al., Model-Driven Software Engineering in Practice, 2012

Ein überaus bekanntes Modell



Niels Bohr (1885 – 1962)



Verkürzungsmerkmal

"Modelle erfassen im allgemeinen n i c h t a l l e Attribute des durch sie repräsentierten Originals, sondern nur solche, die den jeweiligen Modellerschaffern und/oder Modellbenutzern relevant erscheinen"

Abbildungsmerkmal

"Modelle sind stets Modelle v o n e t w a s, nämlich Abbildungen, Repräsentationen natürlicher oder künstlicher Originale, die selbst wieder Modelle sein können."

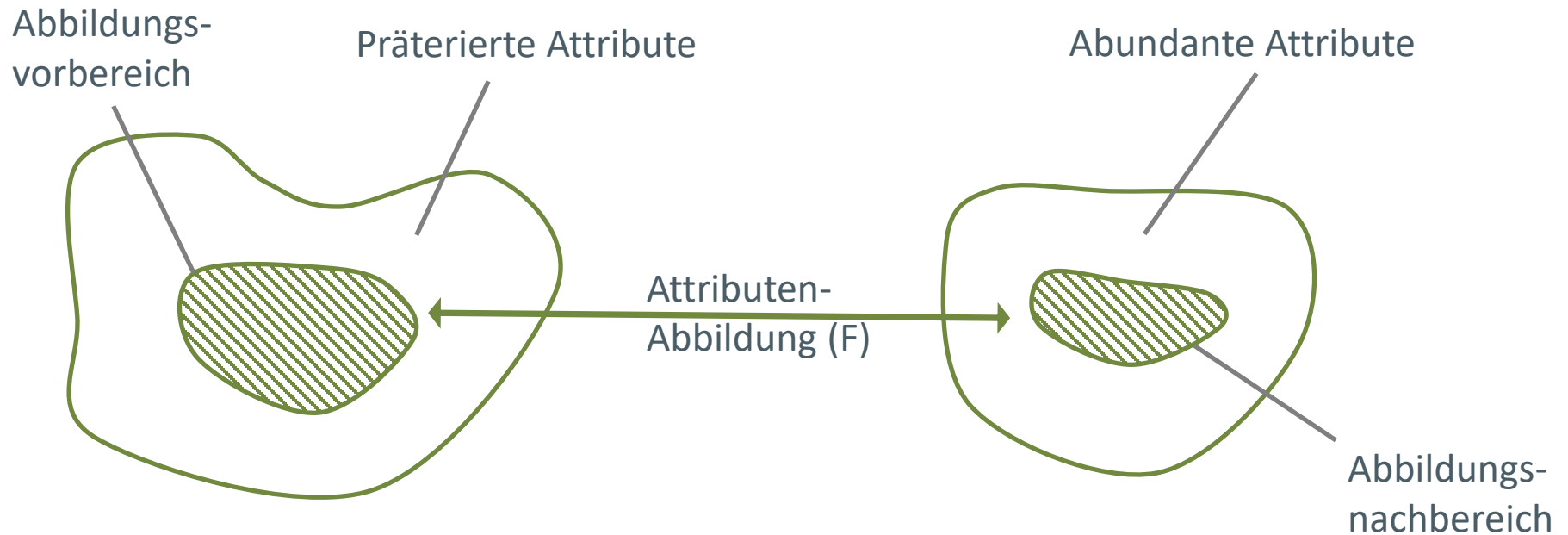
Pragmatisches Merkmal

"Modelle sind ihren Originalen nicht per se eindeutig zugeordnet. Sie erfüllen ihre Ersetzungsfunktion a) für b e s t i m m t e – erkennende und/oder handelnde, modellbenutzende – S u b j e k t e, b) innerhalb b e s t i m m t e r Z e i t i n t e r v a l l e und c) unter Einschränkung auf bestimmte gedankliche oder tatsächliche Operationen."

Stachowiak, Herbert (1973): Allgemeine Modelltheorie. Wien: Springer.

"Über die abbildungsmäßige Originalbezogenheit hinaus ist mithin der allgemeine Modellbegriff *dreifach pragmatisch zu relativieren*. Modelle sind nicht nur Modelle *von etwas*. Sie sind auch Modelle *für jemanden*, einen Menschen oder einen künstlichen Modellbenutzer. Sie erfüllen dabei ihre Funktionen *in der Zeit*, innerhalb eines Zeitintervalls. Und sie sind schließlich Modelle *zu einem bestimmten Zweck*. Man könnte diesen Sachverhalt auch so ausdrücken: Eine pragmatisch vollständige Bestimmung des Modellbegriffs hat nicht nur die Frage zu berücksichtigen, *wovon* etwas Modell ist, sondern auch, *für wen*, *wann* und *wozu* bezüglich seiner je spezifischen Funktion es Modell ist. " (Stachowiak, S. 133)

- für wen
- wann
- wozu



nach: Stachowiak, Herbert (1973): Allgemeine Modelltheorie. Wien: Springer, S. 157

vgl. auch Schalles, Christian (2013): Usability evaluation of modeling languages. @Ireland, Cork Institute of Technology, Diss., 2012. Wiesbaden: Springer Gabler, S. 10 — 11.

Diskutieren Sie!

- Erläutern Sie die drei wesentlichen Modellmerkmale an diesem Beispiel!
- Erläutern Sie Abbildungsvorbereich, präterierte und abundante Attribute sowie Abbildungsnachbereich



Quelle:
https://upload.wikimedia.org/wikipedia/commons/thumb/9/97/The_Earth_seen_from_Apollo_17.jpg/800px-The_Earth_seen_from_Apollo_17.jpg



Quelle:
<https://s3.eu-central-1.amazonaws.com/kosmos.de/media/image/73/94/29/9783440470015.jpg>

Modelle als Skizzen

- Verwendung zum Zwecke der Kommunikation
- Lediglich Teile des Systems sind modelliert

Modelle als Blaupausen

- Modelle bieten eine vollständige und detaillierte Sicht auf das System

Modelle als Programme

- Modelle treten an die Stelle von Programmcode zur Erstellung des Systems

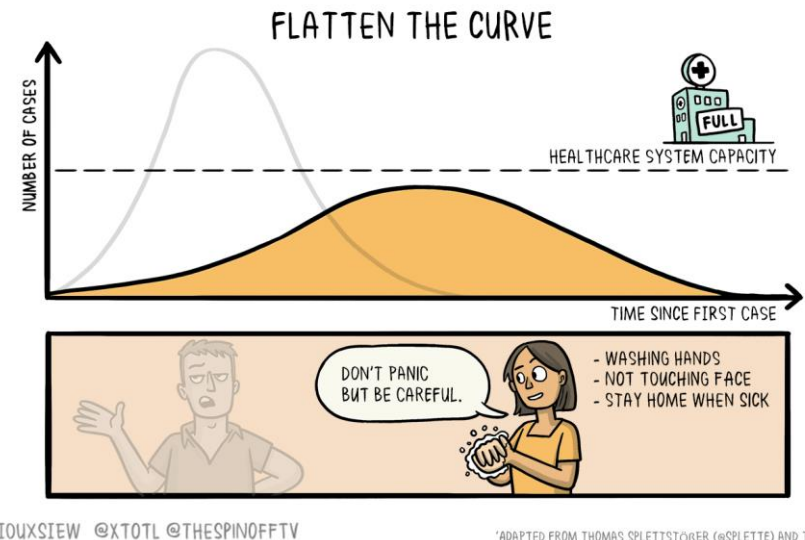
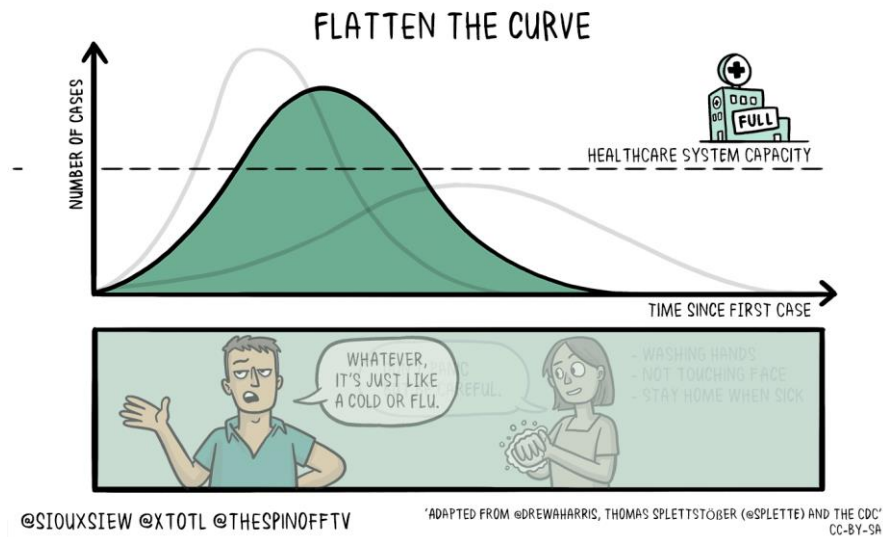
vgl. Brambilla, Marco; Cabot, Jordi; Wimmer, Manuel (2012): Model-driven software engineering in practice. S. 3
vgl. auch <https://martinfowler.com/bliki/UmlMode.html> (letzter Besuch: 11.03.2019)

- Deskriptive Modelle
 - veranschaulichen bereits existierende Systeme
 - abstrahieren Eigenschaften, die nicht von Interesse sind
 - betonen Eigenschaften, die im Fokus des jeweiligen Kontexts stehen
 - werden zur Diskussion, Kommunikation und Analyse eines Systems eingesetzt
- Preskriptive Modelle
 - können für die automatisierte Generierung eines Systems verwendet werden
 - weisen einen hohen Grad an Präzision, Formalität, Vollständigkeit und Konsistenz auf
 - bilden die Basis für modellgetriebene Softwareentwicklung

vgl. Benz, Sebastian; Völter, Markus (2013): DSL engineering. Designing, implementing and using domain-specific languages. [S.l.]: CreateSpace Independent Publishing Platform, S. 31

Diskutieren Sie!

■ Preskriptiv oder deskriptiv?



- Unter Model Driven Software Engineering (MDSE) wird eine Menge von Instrumenten und Richtlinien subsumiert.
- MDSE zielt darauf, Softwareentwicklung von den möglichen Vorteilen des Modellierens profitieren zu lassen
- Bestandteile sind
 - Konzepte: Modelle und Transformationen
 - Notationen: Modellierungssprachen (z.B. MOF, Ecore, UML, etc.)
 - Prozesse und Regeln: Anleitungen, welche Modelle zu welchem Zeitpunkt zu erstellen sind.
 - Tools: Werkzeuge, die
 - das Erstellen der Konzepte unter Verwendung geeigneter Notationen unterstützen.
 - das Verwalten der Artefakte unterstützen
 - die Einhaltung der Prozessabläufe sowie der definierten Regeln unterstützen.

vgl. Brambilla, Marco; Cabot, Jordi; Wimmer, Manuel (2012): Model-driven software engineering in practice. p. 7

MDD – Model-Driven Development

- Modelle sind die bestimmenden Artefakte
- Die Implementierung des Systems geschieht (semi-)automatisiert auf Basis der Modelle

MDA – Model-Driven Architecture

- MDSD-Variante definiert von der OMG
- Verwendet OMG-Standards (u.a. für Notation der Modelle und Transformationen)

MDE – Model-Driven Engineering

- Umfasst zusätzliche modellgetriebene Aufgaben, die über die Implementierung des Systems hinausgehen
- Model-driven system evolution
- Model-driven reverse engineering

vgl. Brambilla, Marco; Cabot, Jordi; Wimmer, Manuel (2012): Model-driven software engineering in practice. p. 9

MBD – Model-Based Development

- Aufgeweichte Version von MDD
- Modelle sind wichtige Artefakte, werden aber eher als Skizzen oder Blaupausen denn als ausführbare Artefakte genutzt

MBE – Model-Based Engineering

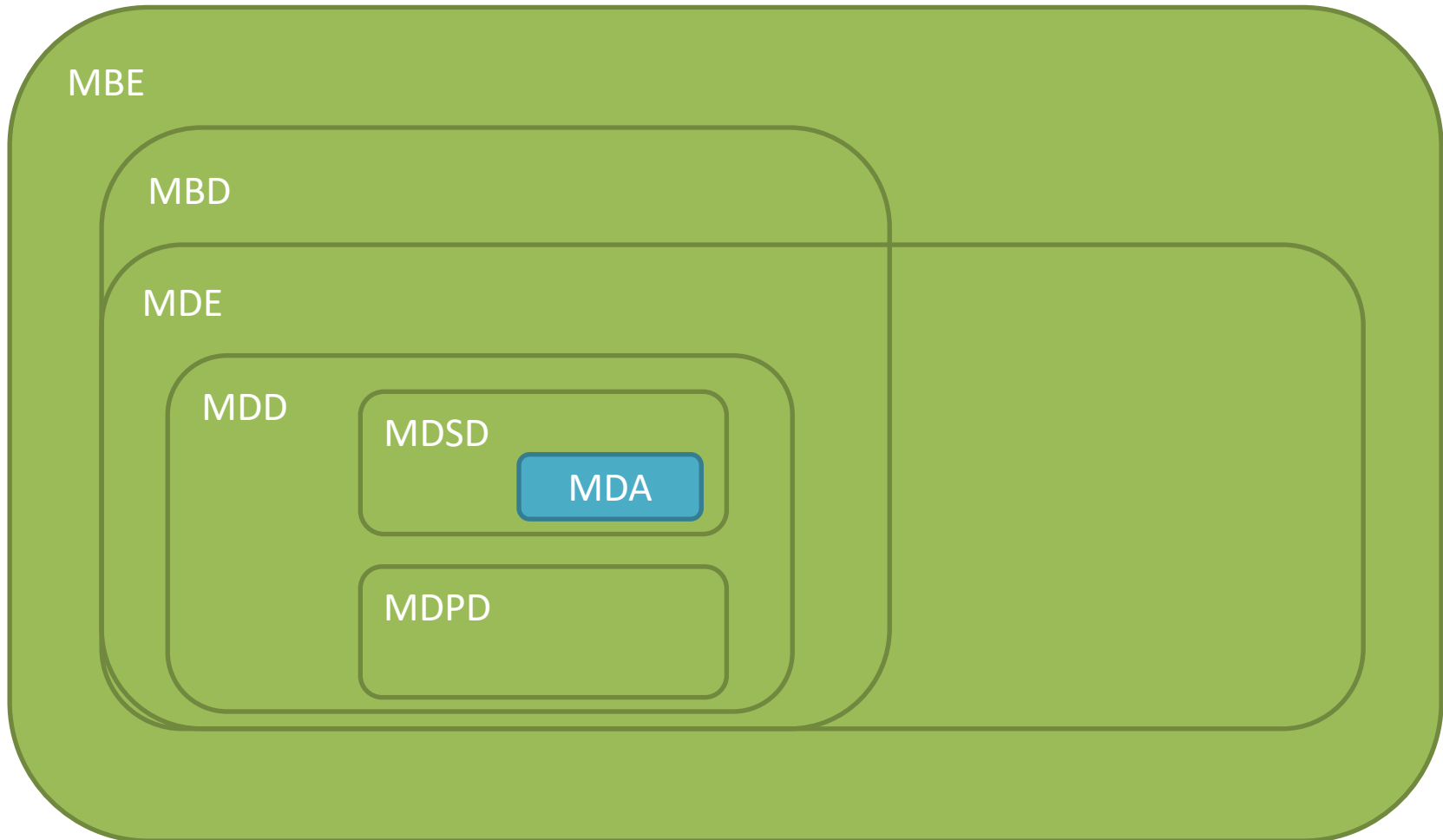
- Aufgeweichte Version von MDE
- Modelle sind wichtige Artefakte, werden aber eher als Skizzen oder Blaupausen denn als ausführbare Artefakte genutzt

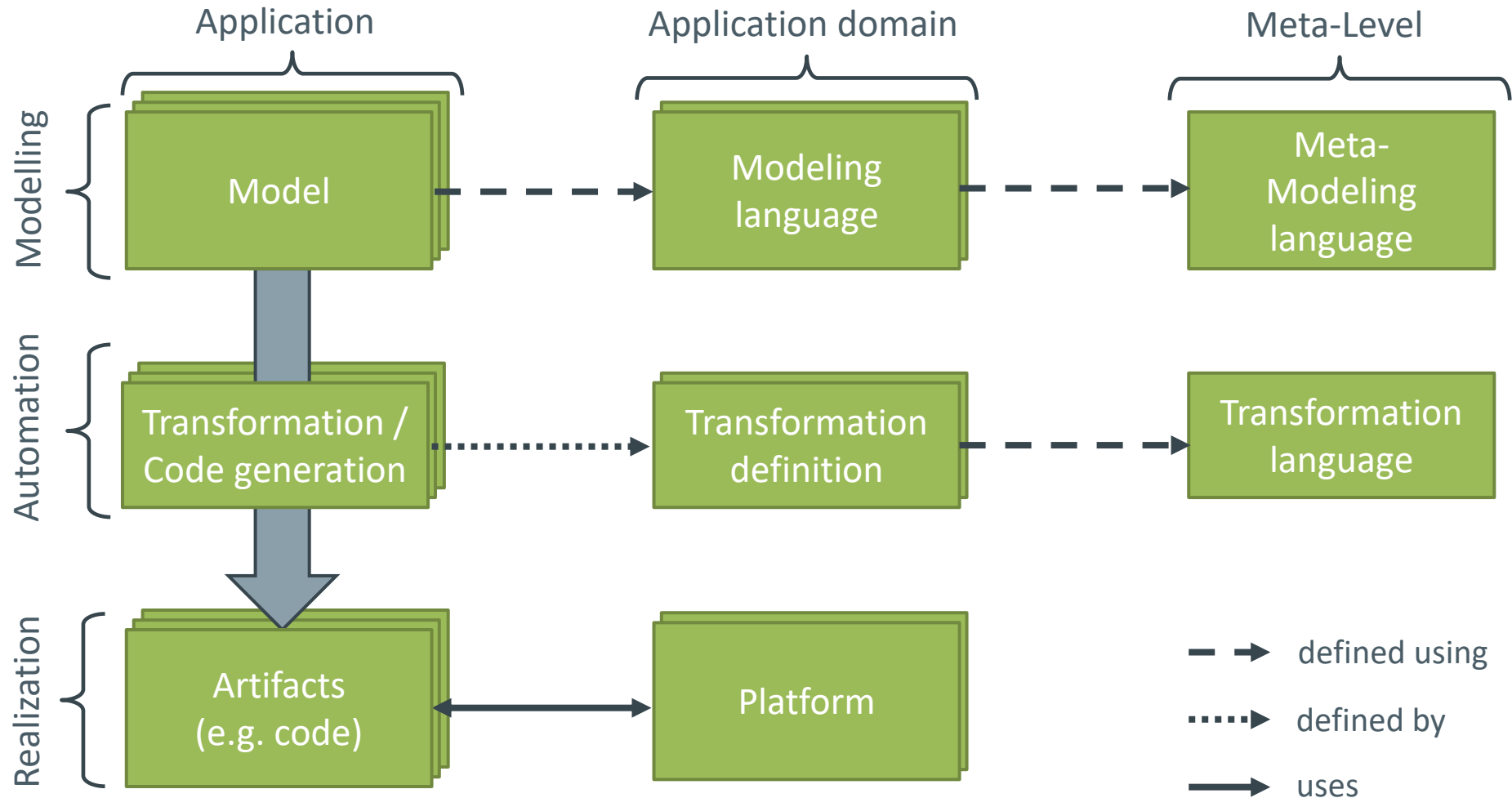
MDSE – Model-Driven Software Engineering

- Modelle werden zur (semi-)automatisierten Generierung von Software genutzt
- Subsummiert unter MDE

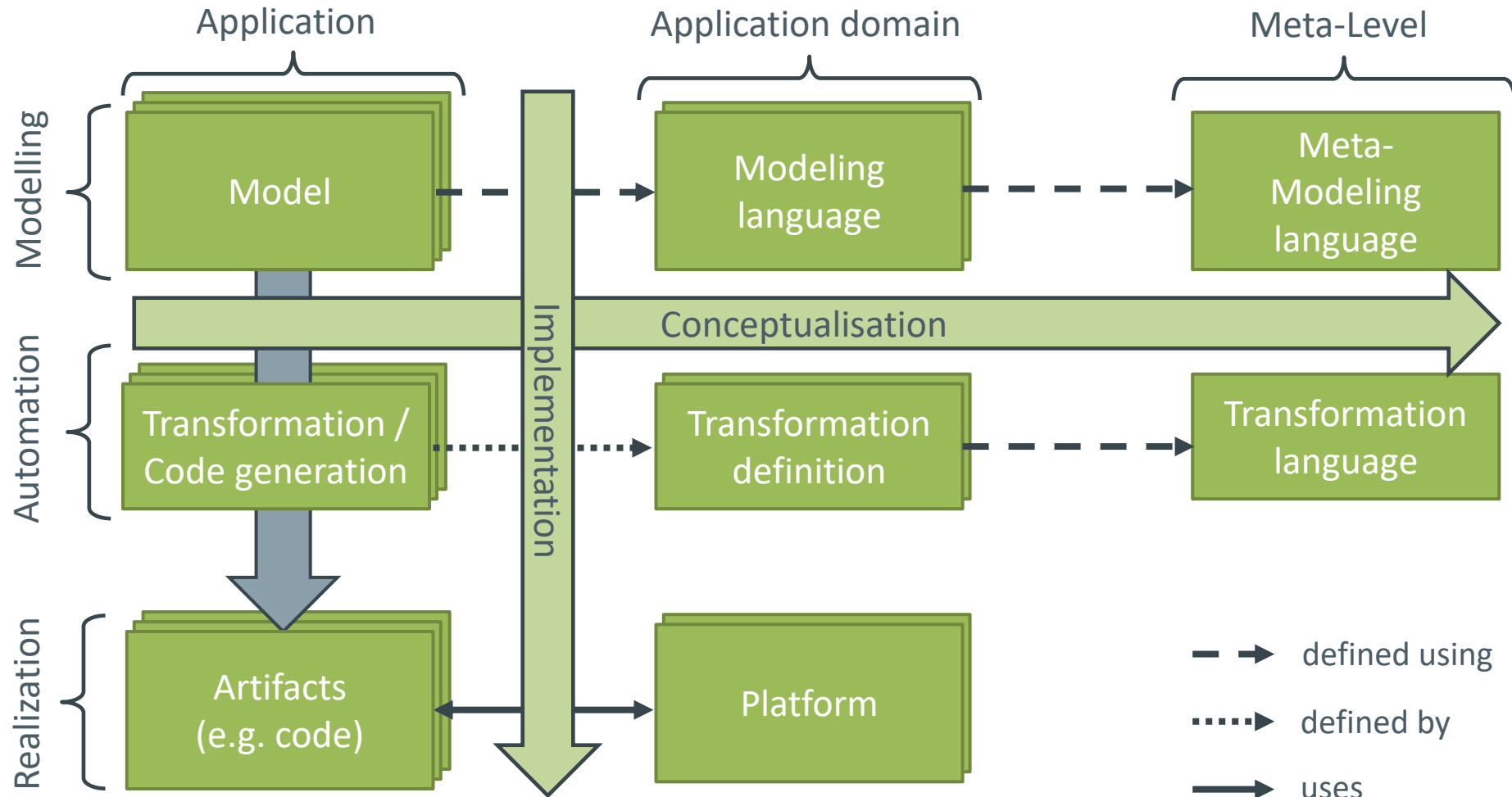
vgl. Brambilla, Marco; Cabot, Jordi; Wimmer, Manuel (2012): Model-driven software engineering in practice. p. 9

Ein Dschungel voller Akronyme





nach Brambilla, Marco; Cabot, Jordi; Wimmer, Manuel (2012): Model-driven software engineering in practice. p. 10



nach Brambilla, Marco; Cabot, Jordi; Wimmer, Manuel (2012): Model-driven software engineering in practice. p. 10

"Modellgetriebene Softwareentwicklung (Model Driven Software Development, MDSD) ist ein Oberbegriff für Techniken, die aus formalen Modellen automatisiert lauffähige Software erzeugen."
(Stahl.2007, S. 11)

- formale Modelle
- lauffähige Software
- automatisiert

Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl.

- In Projekten werden zahlreiche Arten von Modellen eingesetzt
 - Architekturskizzen an Tafeln, Whiteboards, Hand-outs
 - UML-Diagramme zur Herausarbeitung eines bestimmten Design-Aspekts
 - ER-Diagramme als konzeptioneller Entwurf der Datenbank
 - Relationen-Modell zur logischen Beschreibung der Datenbank
 - Flussdiagramme zur groben Beschreibung von Prozessen
 - etc.
- Nur eine Teilmenge der o.g. Modelle ist tatsächlich formal genug, um einsetzbar für MDSD zu sein.
- Formal
 - vollständige Beschreibung der wesentlichen Elemente eines Teilaspektes der Software (vollständig != alles)
 - klare Regeln, über was ein Modell Aussagen treffen kann und muss
 - klare Regeln, wie die Beschreibung erfolgt
 - formale Modelle müssen nicht zwingend UML-Modelle sein!

Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S.11 f.

- Am Ende des MDSD-Prozesses steht ein ausführbares Softwareprodukt.
- Modellierung zum Zwecke der manuellen Implementierung entspricht NICHT dem Gedanken von MDSD.
- Grundsätzliche Unterscheidung von zwei Ansätzen.
- Generatoren generieren aus Modellen Quelltexte anderer Sprachen und sind Bestandteil des Build-Prozesses.
- Interpreter lesen Modelle zur Laufzeit ein und führen korrespondierende Aktionen aus.
- In jedem Fall entsteht aus einem Modell ein Stück ausführbare Software.

Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 12

- Nochmal: Modelle sind nicht bloß eine Spezifikation für manuelle Implementierung!
- Modelle treten an die Stelle des "Quellcodes". die generierten Artefakte bestehen nur temporär.
- Dementsprechend ist es möglich, einen Build-Prozess auf Basis der Modelle durchzuführen.
- ABER: Modelle müssen nicht zwingend das komplette System beschreiben, weshalb auch nicht das komplette System auf Basis des Modells generiert werden muss.
- Eine Kombination aus manuellem und generiertem Code ist durchaus üblich.

Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 13

- Erhöhung der
 - Softwarequalität
 - Wiederverwendbarkeit
 - Entwicklungseffizienz
- Arbeit erfolgt auf höherem Abstraktionsniveau.
- Modellelemente stellen häufig größere Bausteine dar, als sie von der Sprache der Zielplattform geboten werden.
- Die Komplexität wird in zwei Teile gesplittet
 - Definition einer Abbildung des Modells auf die Zielsprache/Zielplattform (Generierung/Interpretation)
 - Bau eines geeigneten Modells, das von der Abbildung automatisiert verarbeitet wird.
 - Technische Details sind bei Erstellung eines Modells transparent für den Modellierer
- Modelle bzw. Modellelemente werden auf gleiche Weise auf eine somit einheitliche Architektur abgebildet.
- Abstraktere Modelle dienen immer auch als Dokumentation.

Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl. S. 13 f.

- Zunehmende Komplexität zu erstellender Softwareartefakte
 - Diese müssen abhängig vom
 - Profil der jeweiligen Stakeholder
 - Phase des Entwicklungsprozesses
 - Ziel des Projektes
- auf unterschiedlichen Abstraktionsniveaus besprochen werden.
- Steigende Durchdringung von Software im Alltag, ständige Weiterentwicklung bestehender Softwareartefakte.
 - Knappheit an Entwicklern mit hervorragenden Kenntnissen in den benötigten Technologien.
 - Interaktion mit Personen, die über wenig/keine Entwicklungserfahrung verfügen (Kunden, Manager), für die ein Modell eine Mediationsrolle für das Systemverständnis einnehmen kann.

vgl. Brambilla, Marco; Cabot, Jordi; Wimmer, Manuel (2012): Model-driven software engineering in practice. [San Rafael, Calif.]: Morgan & Claypool, S. 2 f.

- Expertenwissen (bspw. von Softwarearchitekten) fließt in die Automation ein.
- Wiederverwendbarkeit: Architekturen, Modellierungssprachen und Generatoren können ggf. für unterschiedliche Systeme verwendet werden (Generierung der Persistenzschicht, Generierung der View).
- Plattformunabhängigkeit: Aus den selben Modellen können Artefakte für unterschiedliche Zielplattformen generiert werden (JavaEE, .NET)

Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 13 f.

Ein einführendes Beispiel

- Entwicklung einer Web-basierten Applikation in der Domäne Autovermietung.
- Eingenommene Sichtweise entspricht der eines Anwenders, der den Generator zur Erzeugung der Applikation auf Basis von Modellen verwendet.
- Ein detailliertes Verständnis der zugrunde liegenden Plattformbibliothek ist somit nicht notwendig.



Masken



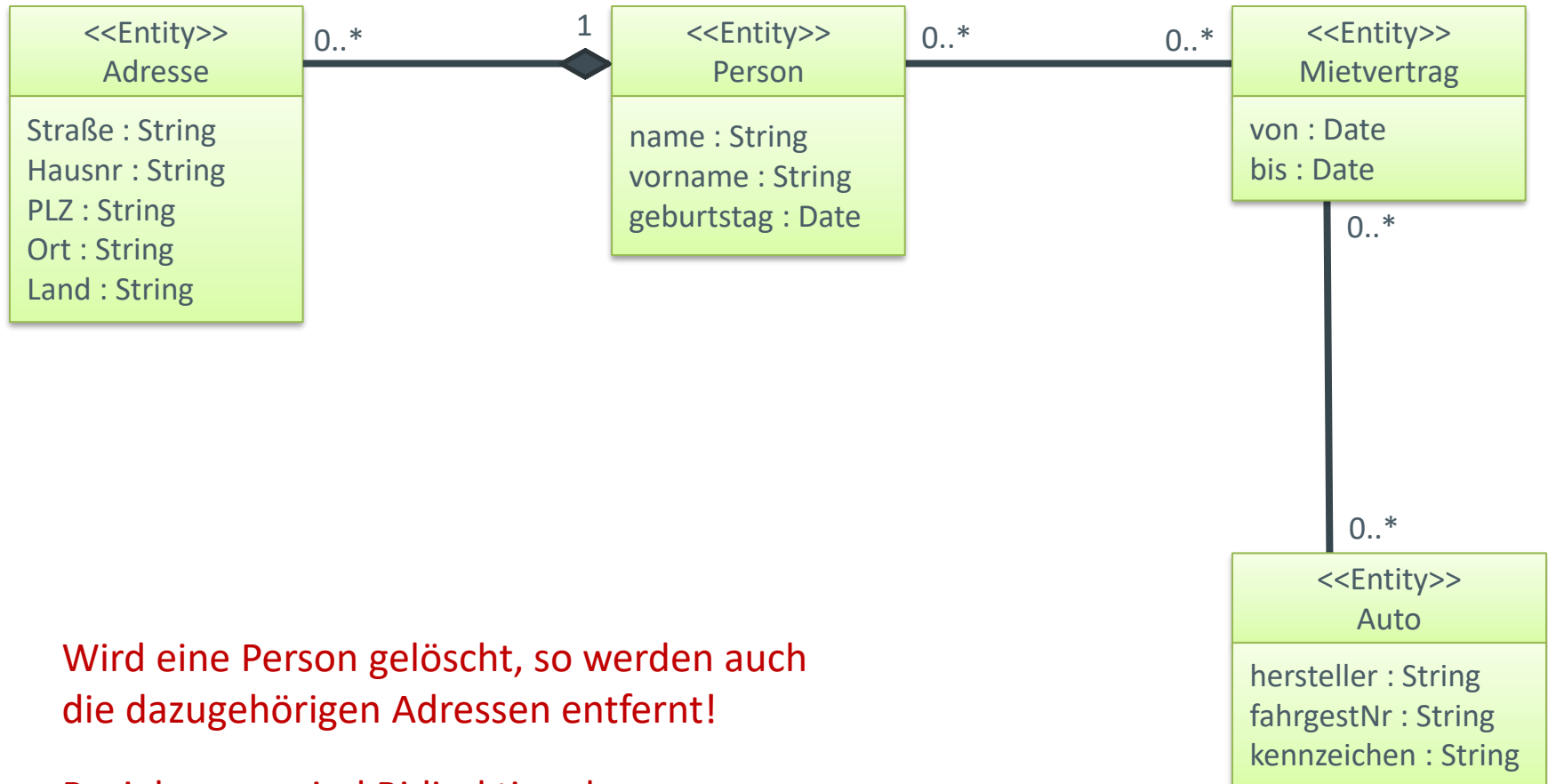
Komponenten



Persistenz

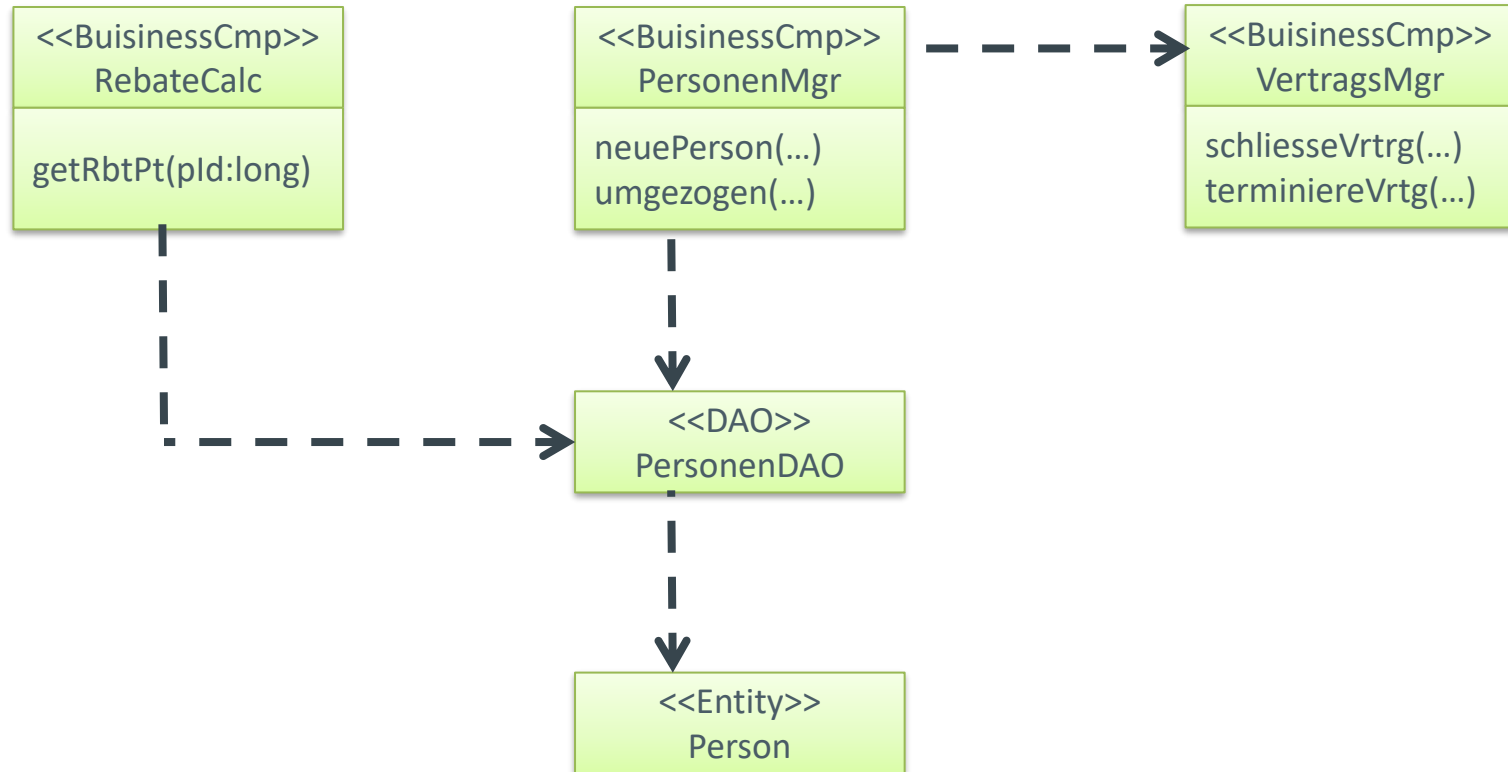


Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 17 – 25

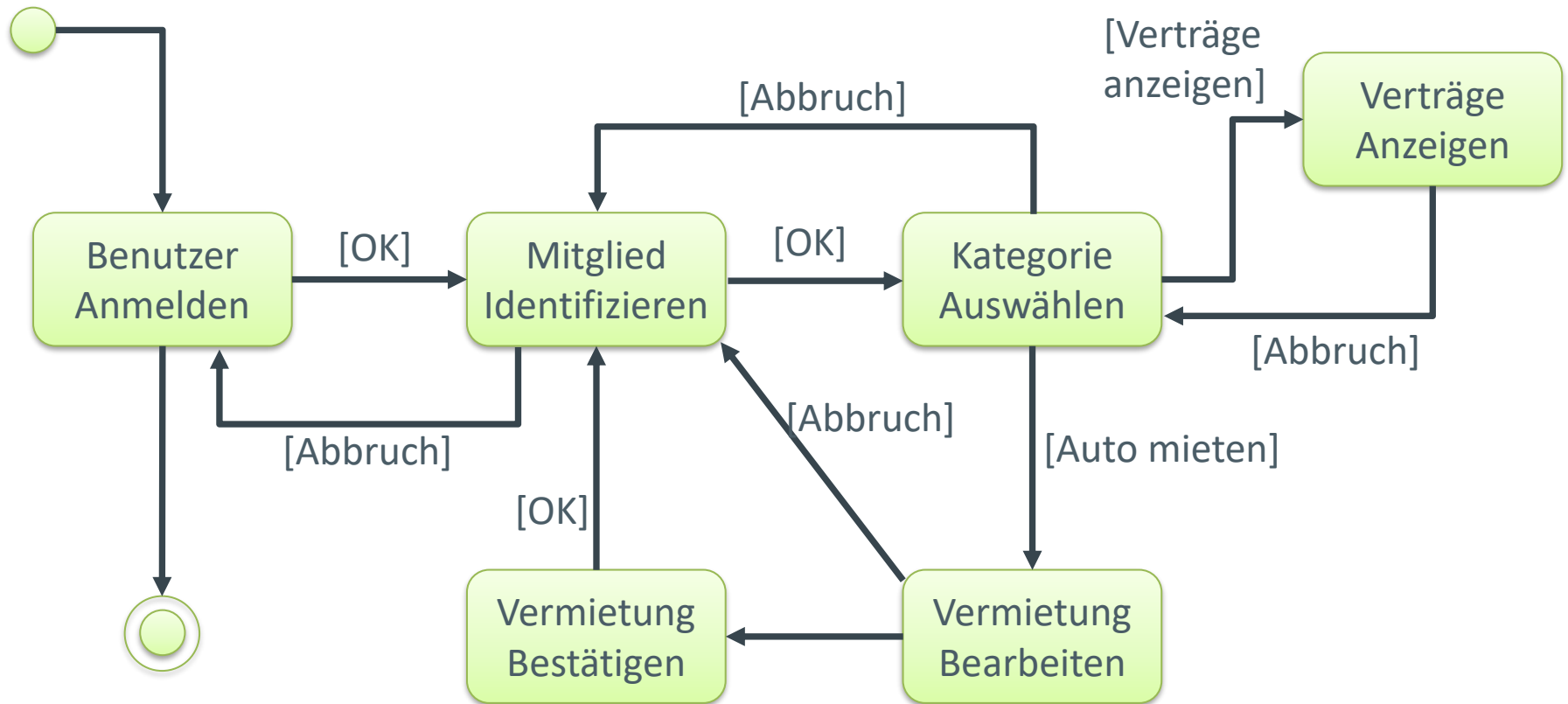


Wird eine Person gelöscht, so werden auch die dazugehörigen Adressen entfernt!

Beziehungen sind Bidirektional.



Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 17 – 25



Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 19

Was wird generiert? Und was nicht?

Entitäten

Für die Entitäten können mit JPA-Annotationen versehene Java-Klassen generiert werden. Hier ist i. d. R. keinerlei manuelle Implementierung nötig.

Komponenten

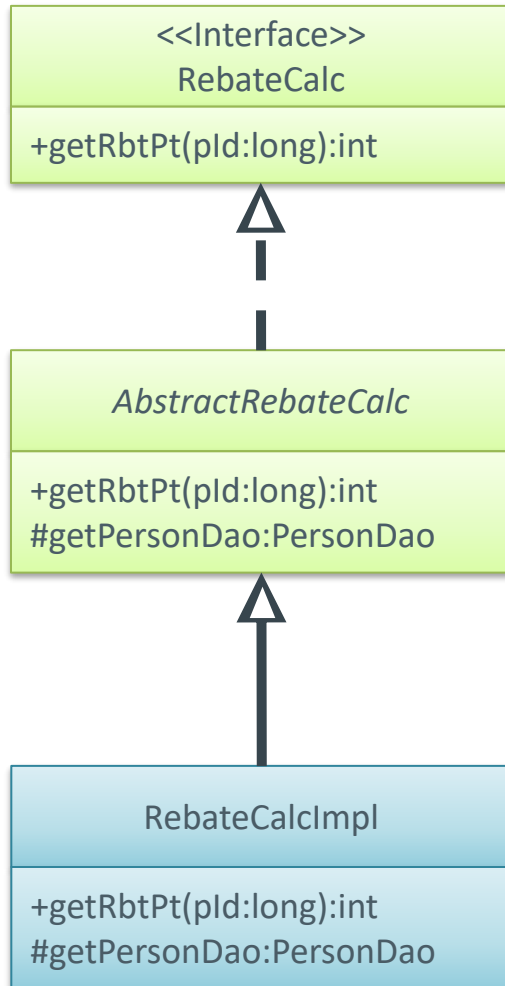
Während die DAO-Komponenten weitestgehend automatisiert generiert werden können, müssen die fachlichen Methoden manuell implementiert werden. Dabei wird häufig eine abstrakte Klasse sowie ein Interface generiert. Innerhalb der abstrakten Klasse findet die korrekte Verdrahtung der Komponenten und Implementierung von Methoden zum Zugriff auf weitere Komponenten statt.

Von der Abstrakten Klasse erbt dann eine Implementierende Klasse, in der die fachlichen Methoden manuell ergänzt werden.

Webseiten

Häufig werden die Views noch manuell erzeugt. Jedoch ist auch eine vollständige Generierung möglich. Aus dem vorliegenden Modell kann die Seitennavigation abgeleitet und generiert werden.

vgl.: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 17 – 25



```
public class RebateCalcImpl extends AbstractRebateCalc{
    public int getRbtPt(long pId){
        Person person =getPersonDao().get(pId);
        int result = 0;
        if(person.getContracts().size() >= 10)
            result += 10;
        if(System.currentTimeMillis()
            - person.getBirthday()
            > 25 * 86400 * 360 * 1000L)
            result += 5;

        return result;
    }
}
```

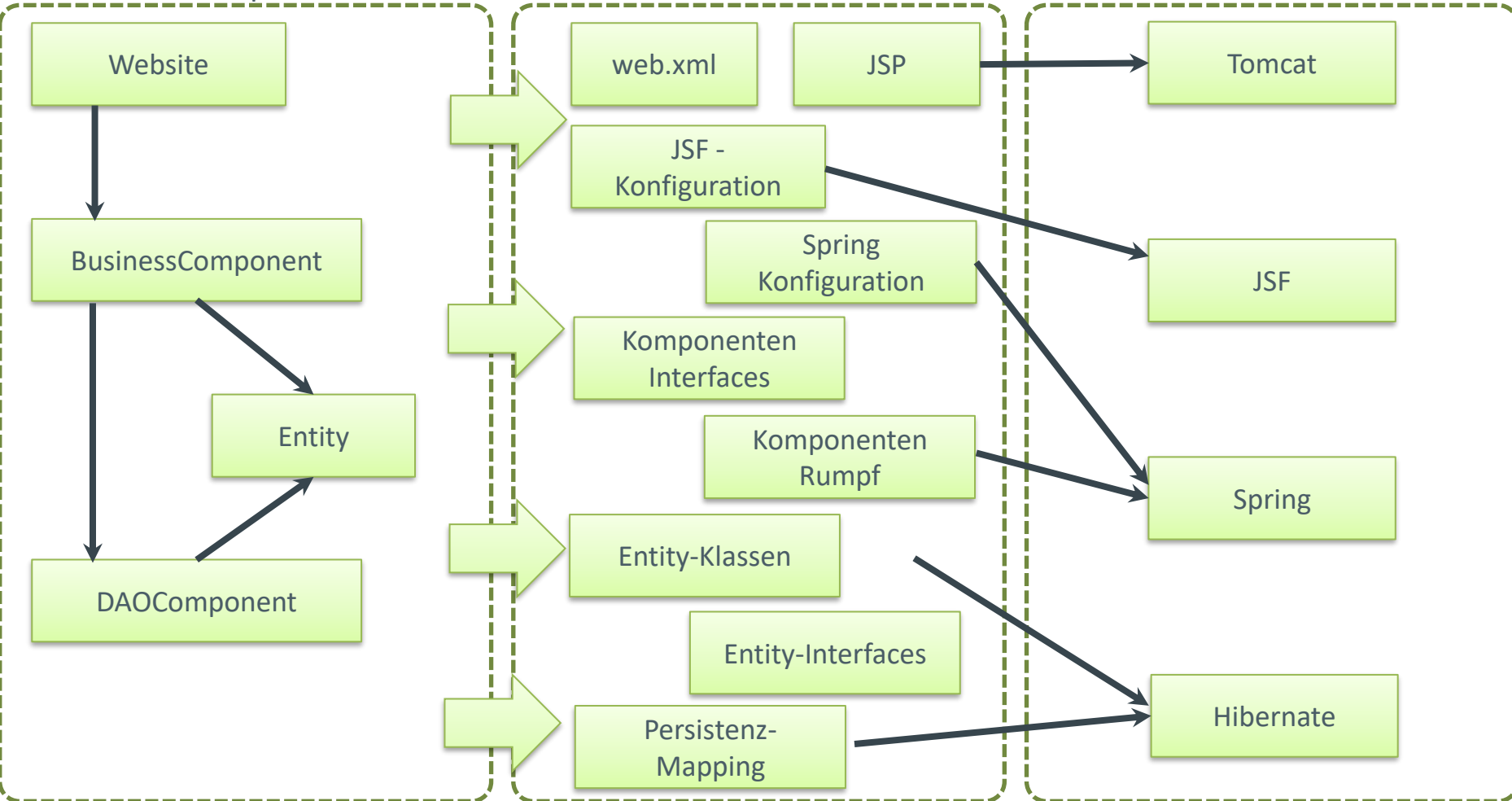
Generierung - Übersicht

➡ = Transformation

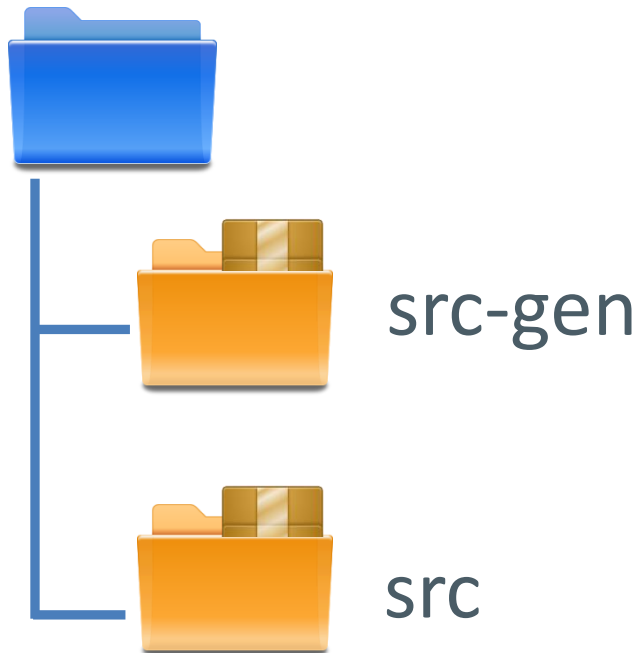
Architekturkonzepte/UML-Profil

Generat

Platform



nach: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 21



- Generierte Klassen
 - Bei jedem Generatorlauf neu erzeugt
-
- Manuell erzeugte Klassen
 - Einmalig generierte Klassen

Templates – But how do it know?

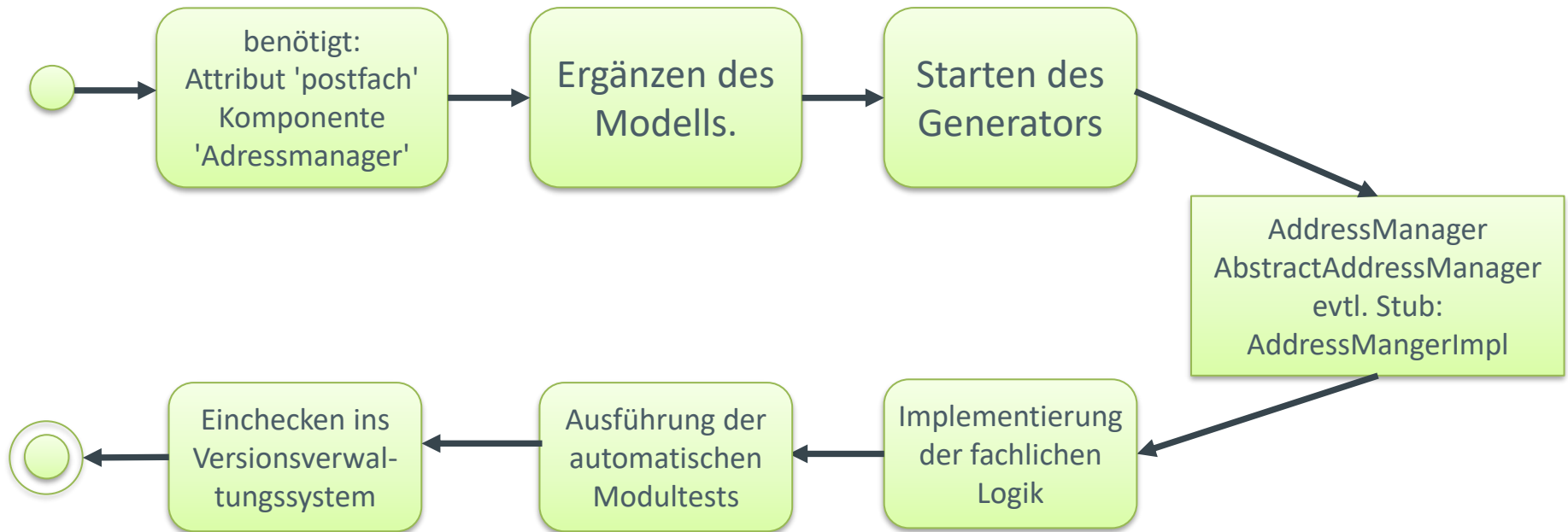
```
«DEFINE BIComponentInterface FOR BusinessComponent»  
  «FILE fullyQualifiedName.replaceAll(".", "/") + ".java"»  
  package «package.fullyQualifiedName»  
  
  import java.util.*;  
  
  public interface «name» {  
    «EXPAND method FOREACH operation»  
  }  
  
  «ENDFILE»  
«ENDDEFINE»  
  
«DEFINE method FOR Operation»  
  «returnType.javaName» «name» (  
    «FOREACH parameters AS param SEPARATOR ", "»  
    «param.type.javaName» «param.type.javaName»  
  «ENDFOREACH»  
  );  
«ENDDEFINE»
```

Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 22

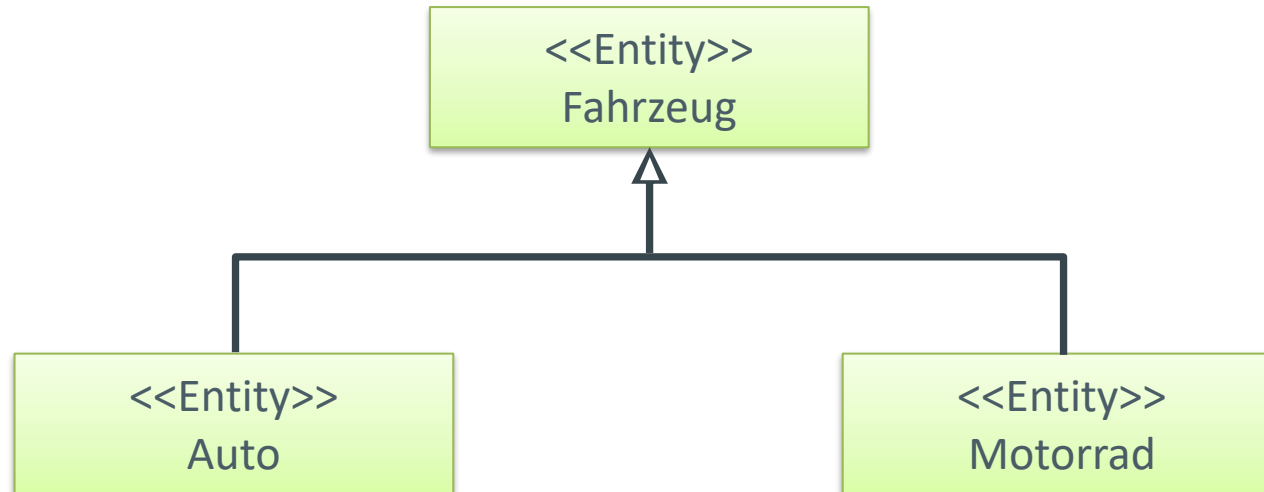
```
package my.demo.pkg;  
  
import java.util.*;  
  
public interface ContractManager{  
    void confirmContract(Contract contract);  
    void extendContract(Contract contract, Date new_endDate);  
}
```

Quelle: Stahl, Thomas (2007): Modellgetriebene Softwareentwicklung. 2., aktualisierte und erw. Aufl. Heidelberg: Dpunkt-Verl., S. 22

- Die Schritte, die auch ohne MDSD-Einsatz erfolgen würden, bleiben bestehen.
- Am Anfang des Build-Prozesses steht mit dem Generatorlauf ein zusätzlicher Schritt.
 - Löschen alter Generate, Erzeugung neuer Generate
- Generierter Code und manuell geschriebener Code beziehen sich aufeinander und ergeben nur zusammen ein funktionierendes Produkt.
- Modelle treten an die Stelle des Quellcodes, da hier die Änderungen vorgenommen werden.



1. Was genau wird in VCS eingecheckt, was nicht?
2. Wie hat sich der Änderungsprozess durch MDSD verändert?



1. Welche Probleme entstehen hier?
2. Wie lassen sich diese lösen?

- Einheitliche Architektur
 - Art, wie Komponenten, Entitäten etc. implementiert sind
- Erhöhter Abstraktionsgrad
 - Modelle näher an der fachlichen als an der technischen Domäne
 - Modelle erlauben einfacheren Überblick über das Gesamtsystem
 - Modelle sind als Ausgangspunkt des finalen Produkts stets aktuell (keine Degeneration der Dokumentation)
- Kapselung von Plattform-Details
 - Details der verwendeten Plattform-Bibliotheken sind in den Transformationen gekapselt.
 - Beim modellieren/Programmieren müssen diese nicht länger berücksichtigt werden.
 - Änderungen der Transformationen erlauben Änderung von Plattformdetails